Exceptions

Writing your own error-handling code, triggering errors, and/or slavishly checking function return values everywhere – all of this gives you some ability to create logic in your code that is dedicated to managing problems. But it's a clunky approach and not what PHP errors are intended for: they're really just for reporting an issue and then either giving up or carrying on. A lot of the time you can deal with some part of your system going *poof* in a cloud of broken logic with some snazzy functions or a class (or two), but how best to reach that magic code? This is where exceptions come in.

Exceptions in PHP are objects, and this chapter appears before Chapter 6, "Object Fundamentals"; you might not yet be comfortable with concepts like classes, objects, methods, or properties. I'll give minimal explanations here concerning the object-oriented parts of the upcoming examples, and you can always refer back to this section once you've absorbed the fundamentals of object orientation in PHP. Or read ahead! It's up to you.

Throwing Exceptions

The mechanisms for using exceptions in PHP are pretty simple (phew!): you create an `Exception` object and `throw` it. Other code can `catch` that object, inspect it, and do whatever is necessary (including throwing it again). An object is just a data structure created from a template called a class, and in this case the class is `Exception` (it's built into PHP's core). You make one with the `new` keyword, then the class name followed by parentheses, and inside the parentheses we put some string data. It looks like a function call, and that's exactly what it is. The `new` keyword triggers a call to a special function called a *constructor*, which deals with the messy business of making a new object and returning a reference to it.

```
throw new Exception("Emergency... there's an emergency going on.");
```

```
Output:
PHP Fatal error:  Uncaught Exception: Emergency... there's an emergency
going on.
```